Logowanie SSH z systemu Windows kartą PIV

Dokument towarzyszący projektowi SSSD Smart Card Profile Manager. Opisuje stronę kliencką: jak zalogować się przez SSH na host, który menedżer już skonfigurował, używając certyfikatu z karty inteligentnej lub tokena USB.

Wersja angielska, `docs/windows-smartcard-ssh.md`, jest źródłem prawdy technicznej. Ten dokument jest jej odpowiednikiem, a nie odgałęzieniem: jeśli któryś zmierzony fakt brzmi w obu inaczej, to jeden z nich jest błędny.

Wszystkie nazwy, odciski i identyfikatory poniżej są **przykładowe**. Podstaw własne.

Okien systemowych nie pokazujemy na zrzutach ekranu — zamiast tego każde jest opisane słowami: tytuł okna, ścieżka w drzewie lub zakładkach, etykiety kontrolek dokładnie tak, jak brzmią, oraz **co powinno się tam zobaczyć, jeśli jest dobrze**. To ostatnie jest najważniejsze.

---

1. Co serwer już zapewnia

Na serwer nie trzeba niczego kopiować. Żaden plik `authorized_keys` nie jest edytowany, żaden klucz publiczny nie jest eksportowany z karty.

Łańcuch po stronie Linuksa wygląda tak:

- Karta zawiera certyfikat X.509 wystawiony przez Twoje CA, na przykład przez `EXAMPLE-Sub-CA`, z nazwą UPN użytkownika (`someone@ad.example.com`) w polu alternatywnej nazwy podmiotu.
- SSSD odwzorowuje ten certyfikat na konto użytkownika według swoich reguł dopasowania i mapowania, a **klucz publiczny SSH wyprowadza wprost z klucza publicznego zawartego w certyfikacie**.
- `sshd` pyta SSSD o ten klucz w momencie logowania, przez dyrektywy poniżej.

```
AuthorizedKeysCommand /usr/bin/sss_ssh_authorizedkeys
AuthorizedKeysCommandUser root
```

Klucz, który serwer zaakceptuje, jest więc funkcją certyfikatu na karcie.

Dwa wnioski warto powiedzieć wprost:

- **Unieważnienie obejmuje także SSH.** Po unieważnieniu certyfikatu, gdy tylko SSSD uwzględni listę CRL, wyprowadzony klucz przestaje być zwracany przez `sss_ssh_authorizedkeys` — i logowanie SSH ginie razem z nim. Nie zostaje żaden nieaktualny wpis w `authorized_keys` do posprzątania. To główny powód, by robić SSH w ten sposób, a nie osobną parą kluczy. Z przekazywaniem agenta z rozdziału 6 rozciąga się to również na `sudo`.
- **Odnowienie nie jest darmowe.** Odnowiony certyfikat zwykle niesie nową parę kluczy, więc wyprowadzony klucz SSH się zmienia. Na serwerze nie trzeba nic edytować, ale odcisk po stronie Windows się zmienia i zapisane rok temu polecenie przestanie działać. Uruchom skrypt pomocniczy ponownie, żeby dostać nowe.

Szybki sposób, by sprawdzić stronę serwera, zanim w ogóle dotkniesz Windows — uruchom na hoście Linux:

```
sss_ssh_authorizedkeys someone@ad.example.com
```

Jeśli wypisze jedną lub więcej linii `ssh-rsa ...`, serwer jest gotowy, a każdy pozostały problem jest po stronie klienta.

---

2. Sprawdzenie stacji roboczej skryptem

`scripts/windows/New-SmartCardSshCommand.ps1` wykonuje wszystkie kontrole opisane niżej i wypisuje gotowe polecenie. Domyślnie jest **tylko do odczytu**: nie uruchamia PuTTY, niczego nie pobiera, nie instaluje sterowników, nie czyta ani nie zapisuje klucza prywatnego i nigdy nie pyta o PIN, nie zapisuje go ani nie loguje. Wymaga PowerShell 5.1 lub 7 i **nie wymaga uprawnień administratora** — wymagają ich tylko proponowane naprawy.

```
.\scripts\windows\New-SmartCardSshCommand.ps1 `
    -Language pl `
    -User someone@ad.example.com `
    -ComputerName linux-host.ad.example.com
```

`-Language pl` przełącza komunikaty na polski. Identyfikatory techniczne nie są tłumaczone — nazwy opcji, ścieżki rejestru, wiersze poleceń, `CAPI:` oraz komunikaty cytowane dosłownie z Windows, OpenSSH czy `certutil` zostają bez zmian, więc polecenie skopiowane z polskiego wyniku jest tym samym poleceniem.

Każda zależność jest raportowana jako PASS / WARN / FAIL z jednoliniową naprawą. Na końcu skrypt wypisuje blok **„Polecenia do uruchomienia na tej stacji roboczej”** — ponumerowaną listę zawierającą wyłącznie te kroki, których ten konkretny przebieg faktycznie wymaga, uporządkowaną tak, by dało się ją wykonać od góry do dołu. Każdy krok da się wkleić osobno; tam, gdzie linie stanowią całość, jest to zaznaczone.

Przydatne parametry:

Parametr	Do czego	
`-Language en\`	pl`	Język komunikatów, domyślnie `en`
`-User`	Nazwa logowania podawana przez `-l`	
`-ComputerName`	Serwer SSH	
`-Reader`	Zawęź do czytników, których nazwa zawiera podany tekst	
`-Thumbprint`	Użyj wskazanego certyfikatu zamiast wybierać z karty	
`-UseOpenSC`	Odczytaj odcisk także przez PKCS#11, jako kontrolę krzyżową	
`-ForwardAgent`	Dodaj `-A` i sprawdź stronę kliencką przekazywania agenta	
`-SessionName`	Zapisana sesja PuTTY, której `AgentFwd`/`TryAgent` zbadać	
`-Launch`	Zastosuj potrzebne ustawienia i połącz się (rozdział 7)	
`-SelfTest`	Sprawdź katalog komunikatów i zakończ	

Kody wyjścia: `0` wypisano polecenie, `1` brak użytecznego certyfikatu, `2` wybór niejednoznaczny, `3` trzeba coś najpierw zainstalować, `4`–`7` błąd przy `-Launch`, `8` brak `-User`/`-ComputerName` przy `-Launch`.

Podłączenie kilku tokenów naraz jest stanem normalnym i skrypt nie zgaduje. Wypisuje, co znalazł, i zatrzymuje się, prosząc o wybór przez `-Reader` albo `-Thumbprint`.

---

3. Trzy pułapki

Te trzy odpowiadają za większość straconego czasu. Każda daje mylący objaw, a dwie nie dają właściwie żadnego komunikatu.

3.1 Klient OpenSSH z Windows w ogóle tego nie potrafi

`C:\WINDOWS\System32\OpenSSH\ssh.exe` — `OpenSSH_for_Windows_9.5p2` — **przyjmuje opcję `-I` i po cichu ją ignoruje.** Obsługa PKCS#11 jest w tej kompilacji wyłączona. Nie ma ostrzeżenia, nie ma błędu, nie ma niezerowego kodu wyjścia, który by o tym powiedział.

Co widać:

```
ssh -I "C:\Program Files\OpenSC Project\OpenSC\pkcs11\opensc-pkcs11.dll" someone@ad.example.com@linux-hos
t.ad.example.com
```

Opcja zostaje przyjęta. Karta nie jest w ogóle dotykana. Uwierzytelnianie przebiega tak, jakby nie zaoferowano żadnego klucza, i kończy się jego brakiem.

Jak to potwierdzić, zamiast zgadywać: uruchom to samo polecenie z `-vvv`. Kompilacja z obsługą PKCS#11 zapisuje w logu ładowanie dostawcy. Ta **nie wypisuje ani linii o dostawcy, ani o** `dlopen` — i właśnie ten brak jest rozpoznaniem.

Wykluczone również: `ssh.exe` dostarczany z Git for Windows (10.5p1) obsługuje PKCS#11, ale załadowanie natywnej `opensc-pkcs11.dll` do tego pliku MSYS2 kończy się powtarzalną awarią. To nie jest obejście.

Czyli: używaj PuTTY-CAC. Nie trać popołudnia na `ssh -I`.

3.2 Tryb X.509 sprawia, że serwer odrzuca klucz

PuTTY-CAC potrafi oferować `x509v3-ssh-rsa` (RFC 6187), gdy jego tryb X.509 jest włączony. **OpenSSH nie implementuje tego algorytmu.** Serwer odrzuca klucz i zapisuje w logu:

```
unsupported public key algorithm
```

Klient tego nie pokazuje. Dostajesz tylko:

```
Server refused our key
```

co wysyła ludzi do sprawdzania `authorized_keys`, reguł mapowania SSSD i uprawnień konta — a żadna z tych rzeczy nie jest problemem.

Naprawą jest `-x509off` przy każdym wywołaniu.

Gdzie to jest w interfejsie. W oknie **PuTTY Configuration**, w drzewie `Category` po lewej, rozwiń `Connection` → `SSH`. Znajduje się tam węzeł `Certificate`, a pod nim trzy gałęzie: `FIDO Tools`, `CAPI Tools` i `X509 Options`. Sam węzeł `Certificate` służy do wybrania certyfikatu; `X509 Options` to miejsce, w którym wyłącza się uwierzytelnianie X.509.

To ustawienie jest **globalne i współdzielone z Pageantem**: gdy raz zostanie włączone, pozostaje włączone przy kolejnych uruchomieniach i dla pozostałych narzędzi pakietu. Dlatego warto podawać `-x509off` jawnie za każdym razem.

Uwaga o stronie `CAPI Tools`, bo nazwa sugeruje coś przeciwnego niż robi. Strona jest zatytułowana `Wizard for managing CAPI certificates` i służy do **tworzenia** certyfikatów samopodpisanych, a nie do wybierania istniejącego. Zawiera sekcję `Creation parameters` z listą rozwijaną `Key Algorithm` (np. `rsa-2048`), polem `Subject Name`, parą przycisków opcji `Provider type` — `Smart Card / Token` oraz `Software` — i przyciskiem `Create Key`. Niżej jest sekcja `Certificate selection filters` z czterema polami wyboru; trzy z nich to `Only Trusted`, `Not Expired` i `Only Smart Card` — etykiety czwartego nie podajemy, bo nie została odczytana na tyle pewnie, by ją tu zapisać. Na końcu `Other options` z `Open Store...` i `Delete Key`. Dla naszego zastosowania użyteczne są **wyłącznie filtry**: magazyn z wieloma certyfikatami zamienia okno wyboru w loterię.

3.3 Odcisk musi pochodzić z karty, nie z magazynu

To najsubtelniejsza z trzech i zarazem kontrola najbardziej warta uruchomienia.

Wpis certyfikatu w magazynie Windows (`Cert:\CurrentUser\My`) wskazuje kontener klucza. Ten kontener może już nie istnieć — karta, do której należał, została ponownie zainicjowana, wymieniona albo zapisana jeszcze raz pod nowym kontenerem. Wpis w magazynie przeżywa; klucz za nim — nie.

Wybierz taki wpis, a Windows powie Ci:

```
Karta inteligentna nie może wykonać żądanej operacji lub operacja wymaga innej
karty inteligentnej
```

To zdanie jest dosłownie prawdziwe, a mimo to bezużyteczne: nie mówi, której karty, a certyfikat wygląda w przystawce MMC zupełnie poprawnie. Klucz, o który system prosi, po prostu nie znajduje się na tej karcie.

Gdzie to widać. Okno systemowe **Windows Security / Karta inteligentna** ma tytuł, pod nim prośbę o wybranie urządzenia karty inteligentnej, a następnie po jednym wierszu na każdego kandydata — w wierszu nazwa karty i nazwa czytnika. Gdy wybrana karta nie może wykonać operacji, pod wierszami pojawia się linia błędu z cytowaną wyżej treścią.

Nieaktualne wpisy gromadzą się szybko. Stacja, na której zainicjowano kilka kart, może spokojnie mieć więcej wpisów martwych niż żywych, a kilka z nich może mieć tę samą nazwę podmiotu — więc wybieranie „tego z moim nazwiskiem” jest rzutem monetą.

Wiarygodny test polega na porównaniu dwóch źródeł wprost. Co karta rzeczywiście zawiera:

```
certutil -scinfo -silent
```

```
Key Container = 00000000-1111-2222-3333-444444444444 [Default Container]
Cert Hash(sha1): aaaaaaaabbbbbbbbccccccccddddddddeeeeeeee
```

Co twierdzi wpis w magazynie:

```
certutil -silent -user -store My aaaaaaaabbbbbbbbccccccccddddddddeeeeeeee
```

```
Key Container = 00000000-1111-2222-3333-444444444444
Provider = Microsoft Smart Card Key Storage Provider
```

Jeśli obie nazwy `Key Container` **się różnią, ten certyfikat jest bezużyteczny**, cokolwiek twierdzi magazyn. Wpis, którego kontenera nie da się w ogóle otworzyć, zgłasza zamiast tego `Missing stored keyset` — to ten sam problem, tylko powiedziany głośniej.

Nazwa kontenera **nie zawsze jest identyfikatorem GUID.** Karty HID Crescendo nadają kontenerom nazwy czytelne dla człowieka, na przykład `Example 9A`. Porównanie nadal obowiązuje; porównuje się napisy, nie GUID-y.

Zwróć uwagę, że `certutil -scinfo` zwraca niezerowy kod wyjścia, gdy jakakolwiek karta w jakimkolwiek czytniku ma problem, mimo że dla zdrowych wypisuje komplet poprawnych danych. **Nie traktuj jego kodu wyjścia jako „brak danych".**

`-silent` powstrzymuje `certutil` od wyświetlenia pytania o PIN. Odczyt certyfikatów i nazw kontenerów nie wymaga PIN-u; wymaga go dopiero samo logowanie SSH.

Certyfikat na karcie to nie to samo co certyfikat w magazynie

Adresowanie `CAPI:` czyta magazyn, więc certyfikat musi tam być zarejestrowany. Usługa propagacji rejestruje certyfikaty karty **tylko przy zdarzeniu włożenia**, więc certyfikat zapisany na karcie, która już tkwiła w czytniku, nigdy nie zostanie zauważony.

Najpierw wyjmij token i włóż go ponownie. To rozwiązuje większość przypadków. Dopiero jeśli to nie pomoże, zarejestruj certyfikat ręcznie — dwa osobne polecenia, wykonywane po kolei:

```
certutil -user -addstore My <cert.der>
```

```
certutil -user -csp "<dostawca, którego -scinfo podaje dla tej karty>" -repairstore My <odcisk-sha1>
```

Napis `Encryption test passed` w wyniku drugiego polecenia oznacza, że powiązanie się udało.

CSP ma znaczenie i jest inny dla każdej karty. Crescendo zgłosiło `Microsoft Base Smart Card Crypto Provider` tam, gdzie YubiKey zgłosił `Microsoft Smart Card Key Storage Provider`. Błędny CSP kończy się błędem wyglądającym jak uszkodzona karta. Skrypt wypisuje to polecenie z już wstawionym CSP, odczytanym z rzeczywistego wyniku `-scinfo`, a nie zgadniętym.

Plik `<cert.der>` trzeba najpierw odczytać z karty; skrypt uruchomiony z `-UseOpenSC` wypisuje gotowe polecenie `pkcs11-tool`, które to robi.

---

4. Polecenie połączenia

```
putty.exe -x509off -i CAPI:<odcisk-sha1> -l someone@ad.example.com linux-host.ad.example.com
```

oraz wariant do użytku skryptowego:

```
plink.exe -x509off -batch -i CAPI:<odcisk-sha1> -l someone@ad.example.com linux-host.ad.example.com
```

Trzy szczegóły nie są oczywiste:

- `-x509off` jest wymagane (rozdział 3.2).
- `-l someone@ad.example.com`, a nie `someone@ad.example.com@host`, ponieważ nazwa UPN w Active Directory zawiera `@`, a składnia `user@host` dzieli się właśnie na tym znaku.
- `<odcisk-sha1>` musi być odczytany **z karty**, a nie wybrany z magazynu Windows (rozdział 3.3).

PuTTY-CAC jest wymagane; zwykłe PuTTY nie obsługuje `CAPI:` w ogóle. Instalacja: `winget install --id NoMoreFood.PuTTY-CAC`.

---

5. Wstępne wczytywanie certyfikatu do Pageanta

Klikanie `Add CAPI Cert` przed każdą sesją szybko się nudzi. Pageant może wstawać przy logowaniu z już wczytanym certyfikatem. Sterują tym trzy wartości, wszystkie bezpośrednio pod kluczem głównym PuTTY:

Wartość	Typ	Znaczenie
`AutoloadCerts`	REG_DWORD	Przeglądaj cały magazyn certyfikatów przy starcie
`SaveCertListEnabled`	REG_DWORD	Pamiętaj listę między uruchomieniami
`SaveCertList`	REG_MULTI_SZ	Sama lista; wpisy mają postać `CAPI:<sha1>`, małymi literami, bez separatorów

Odpowiadające przełączniki to `-autoload` / `-autoloadoff` oraz `-savecertlist` / `-savecertlistoff`. Etykieta w interfejsie brzmi `Autoload Certs && Keys`.

Pułapka: `-autoload` nie znaczy „wczytaj zapisaną listę”

Ta opcja **przegląda cały magazyn certyfikatów Windows i nadpisuje `SaveCertList` przy każdym starcie.** Zmierzone: na stacji testowej zapisała **18** wpisów, w tym certyfikaty o martwych kontenerach kluczy (`Missing stored keyset`). Skrócenie listy do jednego wpisu i ponowne uruchomienie Pageanta bez argumentów dało **znowu 18** — więc ręczne skracanie listy jest bezcelowe, dopóki `AutoloadCerts = 1`.

To nie jest kwestia porządku. **Agent podaje klucze serwerowi po jednym, a domyślne `MaxAuthTries` w OpenSSH wynosi 6.** Przy pełnym magazynie klucz karty może nigdy nie zostać użyty, a awaria objawia się jako:

```
Server refused our key
```

czyli te same cztery słowa, które daje pułapka X.509 z 3.2 i nieaktualny wpis magazynu z 3.3. Trzy niezwiązane przyczyny, jeden komunikat.

Podejście, które działa

Sprawdzone od początku do końca:

1 Przestań przeglądać magazyn:

```
reg add "HKCU\Software\SimonTatham\PuTTY" /v AutoloadCerts /t REG_DWORD /d 0 /f
```

2 Wczytaj ten jeden certyfikat, podając go jako argument pozycyjny. Przekazany już działającemu Pageantowi, dodaje certyfikat do tego agenta, a nie uruchamia drugiego:

```
pageant.exe CAPI:<odcisk-sha1>
```

3 Spraw, by działo się to przy logowaniu — skrót w `%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup`, którego celem jest `pageant.exe`, a polem `Arguments` jest `CAPI:<odcisk-sha1>`.

Co powinno się potem zobaczyć. Okno **Pageant Key List** (dokładnie taki tytuł) ma listę zajmującą większość okna; każdy wczytany klucz to jeden wiersz: algorytm, długość klucza w bitach, odwołanie do klucza zaczynające się od `CAPI:` i czterdziestu małych znaków szesnastkowych, a dalej podmiot certyfikatu zaczynający się od `CN=`. **Jeśli jest dobrze, lista zawiera dokładnie jeden taki wiersz.**

Pod listą są przyciski w dwóch rzędach: `Add PuTTY Key`, `Add PuTTY Key (Encrypted)`, `Add CAPI Cert`, `Add PKCS Cert`, `Remove`, a pod nimi `Re-encrypt` (wyszarzony, o ile nie ma zastosowania) oraz `Add FIDO Key`. W dolnym rzędzie jest lista rozwijana `Fingerprint Type` — **ustaw ją na** `SHA256` — oraz `Help`, `Copy To Clipboard` i `Close`.

Odcisk pokazywany w tym oknie przy ustawieniu `SHA256` musi być **tym samym**, który serwer wyprowadza z certyfikatu. `Copy To Clipboard` pozwala go skopiować i porównać z wynikiem `sss_ssh_authorizedkeys` na hoście.

Jeden przełącznik, który lepiej zostawić

`-forcepincache` trzyma PIN karty w pamięci agenta, więc każdy kolejny podpis — każde logowanie SSH, każde `sudo` przez przekazanego agenta — przechodzi bez pytania. Wygodne na pokaz, ale sprawia, że karta przestaje być drugim składnikiem uwierzytelniania dla każdego, kto dotrze do odblokowanej sesji. Zdecyduj o tym świadomie, a nie przypadkiem.

---

6. Przekazywanie agenta i `sudo`

`sudo` na hoście Linux może uwierzytelnić się wobec karty inteligentnej, mimo że karta tkwi w kliencie Windows. `pam_ssh_agent_auth` weryfikuje podpis złożony przez przekazanego agenta SSH.

Co serwer już zapewnia

W `/etc/pam.d/sudo`, wstawione **przed** `@include common-auth`:

```
auth    sufficient  pam_ssh_agent_auth.so authorized_keys_command=/usr/bin/sss_ssh_authorizedkeys
```

W `/etc/sudoers.d/91-ssh-agent-auth`:

```
Defaults env_keep += "SSH_AUTH_SOCK"
```

Oraz pakiet `libpam-ssh-agent-auth` (w Ubuntu 24.04: noble/universe, 0.10.3-8).

Linia `env_keep` nie jest opcjonalna. `env_reset` w `sudo` domyślnie usuwa `SSH_AUTH_SOCK`; moduł nie znajduje wtedy agenta, a ponieważ linia jest `sufficient`, a nie `required`, stos **po cichu przechodzi do pytania o hasło**. Nic nie zgłasza błędu.

Fragment `authorized_keys_command=` jest architektonicznie najważniejszy. Klucze pochodzą z tego samego źródła AD, którego używa już `sshd`, więc **unieważnienie certyfikatu odbiera także dostęp do** `sudo`. Większość poradników dla tego modułu każe wkleić klucz publiczny do `~/.ssh/authorized_keys`. Nie rób tego: klucz wklejony tam przeżywa unieważnienie i po cichu znosi jedyną właściwość, dla której cały ten układ ma sens.

Dwie rzeczy, które klient musi mieć dobrze

1. Pageant musi działać z wczytanym certyfikatem w momencie połączenia. Przekazywanie agenta jest negocjowane przy otwieraniu sesji. Wczytanie certyfikatu do Pageanta po zestawieniu sesji nie doda go do tej sesji — trzeba połączyć się ponownie.

2. Przekazywanie agenta musi być rzeczywiście włączone. Albo podaj `-A` w wierszu poleceń, albo włącz je dla zapisanej sesji. W interfejsie: w oknie **PuTTY Configuration** przejdź w drzewie `Category` do `Connection` → `SSH` → `Auth` → `Credentials`; tam znajduje się pole wyboru `Allow agent forwarding`. **Jeśli jest dobrze, pole jest zaznaczone, a sesja zapisana ponownie** — inaczej ustawienie zniknie przy następnym uruchomieniu z interfejsu.

Ustawienia dla pojedynczej sesji leżą tutaj:

```
HKCU\Software\SimonTatham\PuTTY\Sessions\<nazwa-sesji>
    AgentFwd   REG_DWORD   1      przekazuj agenta w tej sesji
    TryAgent   REG_DWORD   1      w ogóle używaj Pageanta (domyślnie 1, gdy brak)
```

`-A` w wierszu poleceń załatwia pojedyncze uruchomienie niezależnie od `AgentFwd`; zapisana wartość ma znaczenie przy starcie sesji z interfejsu.

Czym płacisz za `-A`

To jedyna realna cena tej techniki i warto ją powiedzieć wprost.

Dopóki sesja żyje, każdy, kto ma roota na tym serwerze, może użyć przekazanego agenta, by uwierzytelnić się jako Ty wobec innych hostów. Agent podpisuje to, o co zostanie poproszony, a root po drugiej stronie dosięga przekazanego gniazda. Nie może wydobyć klucza prywatnego — ten nigdy nie opuszcza karty — ale nie musi: może pożyczyć Twoją tożsamość na tak długo, jak pozostajesz połączony.

Dlatego włączaj przekazywanie agenta **dla pojedynczych sesji**, do hostów, którym ufasz na tyle, by powierzyć im swoją tożsamość. Nigdy jako `Host *` w konfiguracji klienta i nie domyślnie na hostach przesiadkowych, bo to właśnie maszyny najczęściej współdzielone.

---

7. `-Launch`: zastosuj i połącz

Z przełącznikiem `-Launch` skrypt zastosuje konfigurację, którą dany przebieg uznał za potrzebną, sprawdzi ją **odczytując z powrotem**, a następnie uruchomi PuTTY.

```
.\scripts\windows\New-SmartCardSshCommand.ps1 -Language pl -Launch `
    -User someone@ad.example.com -ComputerName linux-host.ad.example.com
```

Każda zmiana jest raportowana w chwili wprowadzenia razem z poleceniem, które ją cofa, a na końcu wypisywane jest zestawienie tych poleceń cofających.

Co `-Launch` może zastosować — wszystko w zakresie użytkownika i wszystko odwracalne: wartości pod `HKCU\Software\SimonTatham\PuTTY` (`AuthX509`, `AutoloadCerts`, `SaveCertList`), `AgentFwd` i `TryAgent` dla sesji podanej

przez `-SessionName`, skrót w Autostarcie, wczytanie certyfikatu do Pageanta oraz rejestrację certyfikatu karty we własnym magazynie użytkownika.

Czego **nie zrobi**, z założenia:

- **Nie zainstaluje oprogramowania.** `winget` pobiera, instaluje i zwykle wymaga podniesienia uprawnień, a to zmiana maszyny, nie Twojej konfiguracji. Jeśli czegoś brakuje, `-Launch` zatrzymuje się **przed** połączeniem, wypisuje polecenie instalacji i mówi wprost, że niczego nie zmieniono.
- Niczego, co wymaga uprawnień administratora. Zawodzi głośno, zamiast pytać o podniesienie uprawnień.
- Nie zapisze nigdzie PIN-u i nie poda `-forcepincache`.

Kolejność jest istotna: zastosuj, odczytaj z powrotem, a połącz się dopiero, gdy ponowna kontrola przejdzie. Łączenie na podstawie „właśnie zapisałem tę wartość" zamiast „odczytałem ją z powrotem" to dokładnie ten rodzaj fałszywego PASS-a, którego ten projekt ma unikać.

---

8. Weryfikacja i co każdy krok naprawdę dowodzi

Każdy z tych kroków dowodzi czegoś innego. Warto wiedzieć czego, bo trzy niezależne przyczyny dają ten sam komunikat `Server refused our key`.

Krok	Czego dowodzi
`sss_ssh_authorizedkeys someone@ad.example.com` na hoście	Że SSSD wyprowadza klucz z certyfikatu i serwer jest gotowy. Nie mówi nic o kliencie.
`certutil -scinfo -silent`	Że karta jest widoczna i jaki certyfikat oraz kontener naprawdę na niej są. Nie mówi, czy magazyn się z tym zgadza.
Porównanie `Key Container` z karty i z magazynu	Że wpis magazynu wskazuje klucz obecny na **tej** karcie. To jedyny krok, który wyklucza pułapkę 3.3.
Okno **Pageant Key List** z jednym wierszem `CAPI:`	Że agent niesie dokładnie jeden klucz, więc `MaxAuthTries` nie wyczerpie się przed jego podaniem.
Odcisk `SHA256` z Pageanta równy temu z serwera	Że klucz w agencie to ten, który serwer zaakceptuje. Bez tego zgodność jest założeniem.
Logowanie SSH kończy się powodzeniem	Że uwierzytelnianie zadziałało. Nie mówi nic o autoryzacji ani o `sudo`.
`ssh-add -l` wewnątrz sesji	Że agent został przekazany i że widać w nim ten sam odcisk.
`sudo -k && sudo -l` bez żadnego pytania o hasło	Że moduł się uwierzytelnił: linia `sufficient` zwiera stos, zanim `pam_unix` w ogóle zostanie osiągnięty. Pytanie o hasło oznacza, że stos przeszedł dalej.

Do diagnostyki dodaj opcję `debug` do parametrów modułu w `/etc/pam.d/sudo`. Potem ją usuń — zapisuje kilkanaście linii na każde wywołanie `sudo`.

Dwa mylące tropy, oba zmierzone

Żaden z nich nie jest problemem, a oba kosztowały realny czas.

`ssh-agent socket has incorrect permissions for owner` nie dotyczy uprawnień. Komunikat był skutkiem źle sformowanej wartości `SSH_AUTH_SOCK` — ścieżka zawierała zbędne znaki — a nie trybu gniazda. Tryb `0755`, taki jaki tworzy `sshd`, jest **poprawny**; `sudo` uwierzytelniło się dokładnie przy nim. **Nie zmieniaj uprawnień gniazda przez `chmod`.** Użyteczna diagnostyka to:

```
printenv SSH_AUTH_SOCK
ls -l "$(printenv SSH_AUTH_SOCK)"
```

Jeśli wartość zmiennej wygląda nieprawidłowo, to jest właśnie odpowiedź.

`pam_ssh_agent_auth: Using default file=/etc/security/authorized_keys` pojawia się w wyniku diagnostycznym **nawet wtedy, gdy używany jest** `authorized_keys_command`. Komunikat podaje domyślną wartość pola `file` niezależnie od tego, skąd naprawdę pochodzą klucze. Linia `Running AuthorizedKeysCommand:` kilka wierszy dalej jest tą, która mówi, że polecenie jest w użyciu.

Jeszcze jedna uwaga o bibliotekach PKCS#11

Biblioteka DLL z `pkcs11` w nazwie nie musi być modułem PKCS#11. Co najmniej jeden producent dostarcza plik `...-pkcs11....dll`, który nie eksportuje żadnych funkcji `C_*`, więc `C_GetFunctionList` nie daje się rozwiązać i każdy klient PKCS#11 nie potrafi go załadować. Jeśli `-i PKCS:...` nie inicjuje się, sprawdź, czy moduł eksportuje `C_GetFunctionList`, zanim obwinisz kartę albo klienta. Modułem znanym jako działający dla PIV jest `opensc-pkcs11.dll` z OpenSC.